

Name :- Rucha Nitin Pawar

Branch:- AI&DS

DIV :- B

Roll NO :- 70

practical Batch :- B4

Aim:- use sequence data types and their associated operations in python .

Questions:-

1. i) what are sequence data types in python?

Sequence data types in Python are data types that store multiple items in an ordered sequence. Each element in the sequence has a position called an index, which allows us to access the elements.

The main sequence data types in Python are:

1. String (str) – A sequence of characters. Strings are immutable, meaning their values cannot be changed.
2. List (list) – An ordered collection of elements that can store different data types. Lists are mutable, so elements can be modified.
3. Tuple (tuple) – Similar to a list but immutable. Once created, its elements cannot be changed.
4. Range (range) – A sequence of numbers commonly used in loops.

ii) name them and explain how indexing and slicing work across different sequence type?

Indexing means accessing elements using their position number. The index starts from 0 for the first element. Negative indexing can also be used to access elements from the end of the sequence.

Slicing is used to obtain a part of a sequence. It is done using start, stop, and step values. It returns a subset of the sequence without changing the original sequence.

2. i) what is the difference between a list and a tuple in python?

A list is a collection of items that is ordered and mutable, which means its elements can be changed, added, or removed after creation. Lists use square brackets [].

A tuple is also an ordered collection of items, but it is immutable, which means its elements cannot be changed after creation. Tuples use parentheses ().

ii) why would you prefer a tuple over a list in real world applications?

Tuples are preferred when the data should not be modified, such as fixed records like coordinates, dates, or database records. Because tuples are immutable, they are safer and help prevent accidental changes. They also use less memory and are slightly faster than lists.

3. i) How are python arrays different from lists?

Python arrays and lists are both used to store multiple values, but they are different in how they store and handle data.

A list is a built-in Python data structure that can store elements of different data types such as integers, strings, and floats in the same list. Lists are flexible and commonly used in Python programs.

An array in Python is provided by the array module. Arrays can store only elements of the same data type, such as all integers or all floats. This makes arrays more memory-efficient compared to lists.

ii) when should you use the array module instead of a list?

Lists can store different data types, while arrays store only one data type. Lists are more flexible and widely used in Python, while arrays are mainly used for numerical data. Arrays use less memory when storing large amounts of similar data.

4. i) Explain how dictionaries work internally in python?

Dictionaries in Python store data as key-value pairs and are implemented internally using a hash table. When a key is inserted, Python converts the key into a hash value using a hashing function. This hash value determines where the key and its value are stored in memory.

ii) Why are dictionary lookups faster compared to lists or tuples ?

Dictionary lookups are faster than lists or tuples because dictionaries use hashing to access elements directly, while lists and tuples may need to search through elements one by one.

5.i) What is the difference between sets and lists in python? In Python, lists and sets are both used to store collections of items, but they differ in key ways. A list is an ordered collection, meaning it keeps elements in the order they are added and allows duplicate values, while a set is an unordered collection that does not maintain any specific order and automatically removes duplicate elements.

ii)How do sets handle duplicate values, and sets useful in practical scenarios? When duplicate values are added to a set, only one instance is kept because sets are designed

to store unique elements only. Sets are useful in practical scenarios where you need to eliminate duplicates, perform fast membership checks, or carry out mathematical operations like union, intersection, and difference, such as finding common items between two datasets or removing repeated entries from a list.

```
s = input("Enter a string: ")

l = list(map(int, input("Enter list values: ").split()))
t = tuple(map(int, input("Enter tuple values: ").split()))

d = {}
n = int(input("Enter number of items in dictionary: "))
for i in range(n):
    k = input("Enter key: ")
    v = input("Enter value: ")
    d[k] = v

print("String:", s)
print("List:", l)
print("Tuple:", t)
print("Dictionary:", d)

print("First character:", s[0])
print("First element of list:", l[0])
print("First element of tuple:", t[0])

print("String length:", len(s))
print("List length:", len(l))
print("Tuple length:", len(t))

print("Reversed string:", s[::-1])
print("List after adding element:", l + [10])
print("Tuple repeated:", t * 2)

print("Dictionary keys:", d.keys())
print("Dictionary values:", d.values())
```

```
Enter a string: python is best
Enter list values: 1 3 44 55 77 33 6 8 9
Enter tuple values: 66 88 3 2 1 8 9
Enter number of items in dictionary: 3
Enter key: "colloge":"MIT"
Enter value: "name":"rucha"
Enter key: "roll no":70
Enter value: 5
Enter key: name
Enter value: rucha
String: python is best
List: [1, 3, 44, 55, 77, 33, 6, 8, 9]
Tuple: (66, 88, 3, 2, 1, 8, 9)
```

```
Dictionary: {'colloge':"MIT": ' "name": "rucha"', '"roll no":70': '5', 'name': '  
First character: p  
First element of list: 1  
First element of tuple: 66  
String length: 14  
List length: 9  
Tuple length: 7  
Reversed string: tseb si nohtyp  
List after adding element: [1, 3, 44, 55, 77, 33, 6, 8, 9, 10]  
Tuple repeated: (66, 88, 3, 2, 1, 8, 9, 66, 88, 3, 2, 1, 8, 9)  
Dictionary keys: dict_keys(['colloge':"MIT"', '"roll no":70', 'name'])  
Dictionary values: dict_values(['name": "rucha"', '5', 'rucha'])
```

Conclusion:

This practical helped to understand different sequence data types in Python such as list, tuple, string, and range. It also demonstrated basic operations like adding, removing, accessing, and slicing elements. These data types are useful for storing and manipulating ordered data efficiently in Python.